Project & SLA Task Tracker: Technical Report

Group 10 | Formative 1 Group Project | Mobile Development

1. Links and team

GitHub repository

Group contribution tracker

Demo video

Team: Becky Alek Malaak, Peniel Nissi Obeng, Bendou Janna Vitalina Sœur, Ariane Itetero

2. Overview

We built a Flutter Android mobile application for a small software team to create tasks, assign tasks to team members, set priorities and deadlines, and track progress.

The application contains eight main screens: Sign In, Home, Dashboard, Task List, Task Details, Create/Edit Task, Team Members, and Profile. The app does not use a backend; task and team-member data are stored locally on the device.

A key feature that differentiates the application is its SLA-based task status system. Each task is automatically classified as On Track, At Risk, Overdue, or Completed based on its priority, deadline, and completion state. The dashboard then uses these statuses to give the team an overview of its workload and progress.

3. Technical decisions

SLA status: Every task is labelled On Track, At Risk, Overdue, or Completed. At Risk is in two ways: either a task is due within 48 hours, or a high-priority task that is still To Do and is due in 3 days. We implemented the second rule to catch important work that nobody has started. The status is calculated every time it is displayed but we don't save it. Deadline times are usually 23:59:59 of that day.

Local storage: We used *shared_preferences* (Google, 2026), and we saved tasks and members as JSON text. Our data is small, so we did not need the tables and queries from SQLite. If saved data is missing or corrupted, the app falls back to sample data instead of crashing.

State: We used *setState()*. A *HomeShell* widget has the lists of tasks and members, and every change follows the same order: build the new list, save it, then call *setState*. The screens will then receive data through their constructors, so there is one clear place where data changes.

Navigation. We used named routes (Flutter, 2026a) to handle moving between screens, and a bottom NavigationBar with IndexedStack switches between the four main tabs.

Validation. Form fields use functions from one *Validators* class (Flutter, 2026b). Every validator function returns an error message or *null*. The rules are: titles should contain 3 to 80 characters even after we trim, description is optional but only up to 500 characters, an assignee and a deadline are

required, a new task's deadline cannot be in the past, names must be unique, and emails must use a valid format.

Teamwork. Each member worked on specific files and screens. We used new branches for new tasks, and every PR was reviewed before being merged into main. We agreed on some model classes and screen constructors at the start to avoid conflicts.

4. Challenges and how we solved them

Becky Alek: While building the foundation, I faced startup crashes and UI freezes because asynchronous data loading in `initState` conflicted with missing team stubs. I traced the cause using terminal stack traces, identifying unhandled futures and type mismatches. I fixed this by adding loading indicators, safe fallbacks, and centralized callbacks in `HomeShell`. This taught me the importance of clear data contracts before building interdependent modules.

Peniel Nissi Obeng: Storage service returned an empty list when nothing was saved, so on first launch, Sign In had no members to pick. A teammate caught it in review. I fixed it by loading sample data whenever nothing is saved. I also had to make sure a task due today isn't marked overdue until the day ends.

Bendou Janna Vitalina Sœur: When a member was opened from the Team screen, my Task List showed only that member's tasks but had no way to show or remove the filter. I found the cause by reading home_shell.dart, where HomeShell filtered the list itself and passed only those tasks to my screen, so my screen did not know a filter was active and could not bring the other tasks back. I fixed it in two steps: I added an optional initialAssigneeId parameter and a removable "Assigned to" chip to the Task List screen, and I asked Person A to pass the full task list plus the member id instead of a pre-filtered list. Person A made the small change in HomeShell, and now tapping the x on the chip shows every task again. I learned to agree early on which part of the app owns each piece of logic, and that a screen should receive the full data and do its own filtering.

Ariane Itetero: My main challenge was fitting the dashboard information on one screen while keeping it easy to read and scroll. I added a progress ring and status cards to show how many tasks are completed, on track, at risk, or overdue. I made sure the progress ring works when there are no tasks, and fixed it when it first appeared too small. I also added a section that shows up to five tasks that need attention, with the soonest deadlines first. On the Team screen, I added task counts, overdue counts, and workload bars for each member, along with a form for adding a new member. I tested these features to check that they worked and that the screens fit on smaller displays.

Shared challenge: merge conflicts. Early on, two branches contained their own copies of the same file, so pull requests reported conflicts. We solved it by agreeing on file ownership: only the owner edits a file, and everyone else asks for changes. We also learned to merge *main* into our branches regularly and to stage files by name instead of committing everything.

5. Testing

A clean Flutter analysis was required before anyone created a PR; reviewers tested branches locally before merging, and we wrote unit tests for the sla logic and the validators. We tested every screen on an Android emulator only, including closing and reopening the app to confirm that data stays.

6. AI Usage Declaration

AI was mainly used to plan the project structure and split tasks. We also used various AI tools for debugging, sample data, a guide for the test files, and reviewing PRs against our plan. Finally, we used AI to aid with the README file and references in this report.

flutter analyze and flutter test commands were always a requirement before anything was pushed. We ran the app on an Android emulator to check the behaviour and UI for any task that worked on screens. Every PR was reviewed before approval, and in fact, some PRs had to be redone in order to get approval.

7. References

Flutter. (2026a). *Navigate with named routes*. Google. https://docs.flutter.dev/cookbook/navigation/named-routes

Flutter. (2026b). *Build a form with validation*. Google. https://docs.flutter.dev/cookbook/forms/validation

Flutter. (2026c). *Store key-value data on disk*. Google. https://docs.flutter.dev/cookbook/persistence/key-value

Google. (2026). *shared_preferences* [Software package]. https://pub.dev/packages/shared_preferences

Material Design. (2026). *Material Design 3*. Google. https://m3.material.io